Assignment 6:

Medians and Order Statistics & Elementary Data Structures

Hoai Nam Tran

(MSCS-532-M80) Algorithms and Data Structures

Dr. Michael Solomon

University of the Cumberlands

1. Introduction

This assignment examines two fundamental topics in computer science: selection algorithms and elementary data structures. In the first part, the goal is to implement and compare a deterministic selection algorithm and a randomized selection algorithm for finding the k^{th} smallest element in an array, along with their theoretical and empirical performance analysis.

In the second part, the assignment focuses on implementing basic data structures such as arrays, matrices, stacks, queues, and singly linked lists. It also requires analyzing the time complexity of their main operations and discussing their practical uses and trade-offs.

Overall, this report explains the implementation, analysis, and results of both parts, showing how algorithm choice and data structure design affect efficiency in solving computational problems.

2. GitHub Repository Link

Part1: Selection Algorithms:

https://github.com/NamTran9694/MSCS532_Assignment6/blob/main/part1_selection.py

Part 2: Data Structures

https://github.com/NamTran9694/MSCS532_Assignment6/blob/main/part2_data_structures.py

Empirical Test:

https://github.com/NamTran9694/MSCS532_Assignment6/blob/main/empirical_test.py

3. Part 1: Implementation and Analysis of Selection Algorithms

3.1. Overview of Selection Algorithms

Selection algorithms are used to find the k^{th} smallest element in an unsorted array without fully sorting it. This problem is important in tasks such as finding medians, percentiles, and ranked values. In this assignment, two approaches were implemented: a deterministic selection algorithm using Median of Medians and a randomized selection algorithm using Randomized Quickselect. These two methods solve the same problem but differ in how they choose the pivot and in their performance guarantees (Cormen et al., 2022).

3.2. Design Choices & Implementation Details

The deterministic algorithm was implemented using the Median of Medians method. The array is divided into groups of five, each group is sorted, and the medians are collected. The median of these medians is then used as the pivot for partitioning. This design was chosen because it guarantees a good pivot and supports worst-case linear time.

The randomized algorithm was implemented using Randomized Quickselect. In this method, a random pivot is selected, and the array is partitioned into elements less than, equal to, and greater than the pivot. This approach is simpler and usually faster in practice. In both implementations, three-way partitioning was used so duplicate values could be handled correctly, and small arrays were handled directly to reduce recursive overhead.

3.3. Theoretical Complexity Analysis

The deterministic Median of Medians algorithm has a worst-case time complexity of $O(n)$ because it always selects a reasonably balanced pivot. Its recurrence is $T(n)=T(n/5)+T(7n/10)+O(n)$, which solves to linear time (Cormen et al., 2022).

The randomized selection algorithm has expected O(n) time complexity because random pivots usually produce balanced partitions on average. However, its worst-case time complexity is $O(n^2)$ if poor pivots are repeatedly chosen. In practice, the randomized version often runs faster because it has lower overhead than the deterministic method.

3.4.Empirical Analysis

To evaluate the practical performance of the selection algorithms, empirical tests were conducted using three different input distributions: random, sorted, and reverse-sorted arrays. For each distribution, the running times of both the deterministic (Median of Medians) and randomized (Randomized Quickselect) algorithms were recorded across input sizes of n=100, n=500, n=1000, and n=5000.

```
Input Type: random
n=100   Deterministic=0.000062s  Randomized=0.000024s
n=500   Deterministic=0.000323s  Randomized=0.000080s
n=1000  Deterministic=0.000697s  Randomized=0.000164s
n=5000  Deterministic=0.003509s  Randomized=0.000820s

Input Type: sorted
n=100   Deterministic=0.000050s  Randomized=0.000015s
n=500   Deterministic=0.000220s  Randomized=0.000066s
n=1000  Deterministic=0.000440s  Randomized=0.000121s
n=5000  Deterministic=0.002250s  Randomized=0.000600s

Input Type: reverse
n=100   Deterministic=0.000061s  Randomized=0.000017s
n=500   Deterministic=0.000343s  Randomized=0.000070s
n=1000  Deterministic=0.000654s  Randomized=0.000136s
n=5000  Deterministic=0.003366s  Randomized=0.000613s
PS C:\Users\nam.tran>
```

Linear Time Scaling (O(n)): For both algorithms across all input types, the execution time scales linearly with the input size. For instance, as the input size increases by a factor of 5 (from n=1000 to n=5000), the execution time for both algorithms roughly increases by a factor of 5. This confirms the O(n) time complexity in practice.

Constant Factor Overheads: The data clearly demonstrates that the randomized algorithm is consistently 3 to 4 times faster than the deterministic algorithm across all test cases. This validates the theoretical expectation that while the Median of Medians algorithm guarantees worst-case linear time, it carries a significant constant-factor overhead. The operations required to divide the array into groups of five, sort those small groups, and recursively find the median of medians add considerable computational weight compared to the simple random pivot selection of Quickselect.

Impact of Input Distribution: The randomized algorithm maintained its superior performance even on sorted and reverse-sorted inputs. A naive Quickselect (picking the first or last element as a pivot) would degrade to $O(n^2)$ on sorted or reverse-sorted data. However, the consistent performance recorded here proves that selecting a random pivot successfully mitigates this risk, yielding the expected O(n) performance regardless of the initial array distribution. Interestingly, both algorithms performed slightly faster on the sorted input compared to the random input, likely due to better branch prediction and cache locality at the processor level.

4. Part 2: Elementary Data Structures Implementation and Discussion

4.1. Overview of Elementary Data Structures

Elementary data structures are the basic tools used to organize and manage data in computer programs. In this assignment, the implemented structures were arrays, matrices, stacks, queues, and singly linked lists. Each structure supports data storage and operations in a different way, which affects efficiency and practical use. Understanding these differences is important because the choice of data structure can directly influence the performance of an algorithm (Goodrich et al., 2013).

4.2.Design Choices & Implementation Details

The data structures were implemented in Python using class-based designs. Arrays and matrices were represented using Python lists, which allowed simple access and update operations. The stack was implemented with push and pop operations following the last-in, first-out (LIFO) rule, while the queue used enqueue and dequeue operations following the first-in, first-out (FIFO) rule. The singly linked list was implemented with nodes, where each node stores data and a reference to the next node.

These implementations were designed to clearly demonstrate the behavior of each structure. For example, arrays provide direct indexed access, while linked lists require traversal. Stacks and queues were kept simple so their core operations could be analyzed easily and compared with other structures.

4.3.Performance Analysis

The performance of each data structure depends on the operation being performed. Arrays allow O(1) access by index, which makes them efficient for retrieval. However, insertion and deletion in the middle usually require shifting elements, giving O(n) time complexity. Matrices also provide constant-time access when row and column indices are known.

Stacks are efficient because both push and pop operations can be performed in O(1) time. Queues support enqueue efficiently, but if implemented with a standard list, dequeue from the front may require shifting elements and can take O(n) time. Singly linked lists support insertion at the beginning in O(1) time, but searching, deletion, or traversal generally requires O(n) time because nodes must be visited one by one (Cormen et al., 2022).

4.4.Practical Applications & Trade-offs

Each data structure is useful in different situations. Arrays are preferred when fast access to elements is needed, such as in tables or fixed collections of values. Matrices are useful for grid-based data, image processing, and numerical computation. Stacks are commonly used in function calls, undo operations, and expression evaluation. Queues are widely used in scheduling, buffering, and breadth-first search. Linked lists are helpful when insertions and deletions happen frequently and direct indexing is less important.

The main trade-off is between speed of access and flexibility of modification. Arrays are fast for indexing but less efficient for insertions and deletions. Linked lists are more flexible for updates but slower for access. Stacks and queues are specialized structures that simplify problems where strict ordering rules are required. Overall, the best choice depends on the type of operations a program performs most often.

5. Conclusion

This assignment demonstrated the importance of both selection algorithms and elementary data structures in solving computational problems efficiently. In Part 1, the deterministic Median of Medians algorithm and the randomized Quickselect algorithm were implemented and compared. The analysis showed that both algorithms are effective for finding the k^{th} smallest element, but they differ in performance guarantees and practical efficiency. The deterministic method provides worst-case linear time, while the randomized method is usually faster in practice but only guarantees expected linear time.

In Part 2, basic data structures including arrays, matrices, stacks, queues, and singly linked lists were implemented and analyzed. Each structure has its own strengths and limitations

depending on the required operations. Arrays and matrices provide fast access, stacks and queues support ordered processing, and linked lists offer flexibility for insertions and deletions.

Overall, this assignment showed that choosing the right algorithm and data structure is essential for improving program efficiency. It also highlighted the connection between theoretical complexity analysis and actual program behavior in practice.

6. Reference

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.